OOP LAB SERIES — PART FIVE

THE COLLECTIONS FRAMEWORK

Building a Contact Manager with ArrayList, LinkedList, HashSet, LinkedHashSet, TreeSet, and HashMap — solving the same growing-contact-list problem by hand first, every time, before the library does it for you.

COURSE	Object-Oriented Programming (Java)
LECTURER	Jacob Chikwanda
DURATION	40 minutes
GROUP SIZE	5 (Driver / Narrator / Bug Hunter / Time Keeper / Scribe)

BUILDS ON WEEK 4 — CONTACT, PRIVATE FIELDS, ILLEGALARGUMENTEXCEPTION

PART 1 — SOLVE IT BY HAND FIRST

WHY WE NEED COLLECTIONS · 6 MIN

DRIVER	types every line below
NARRATOR	reads each step aloud before Driver types it
BUG HUNTER	predicts the output before Enter is pressed
TIME KEEPER	watches the clock, calls out at 20 / 35 min
SCRIBE	fills in the boxes on this page as you go

Before any collection gets a name, do exactly what was done live: build a one-slot contact array, then grow it by hand.

1. Create `Main.java` in a fresh `com.zut` project. Wrap the whole thing in a `try`/`catch` — a Week 4 habit, even though nothing will throw here.

MAIN.JAVA

```
try {
    String[] contact = new String[1];
    contact[0] = "Mary - 091";

    int contactArrayLength = contact.length;
    String[] contacts = new String[contactArrayLength + 1];
    System.arraycopy(contact, 0, contacts, 0, contactArrayLength);
    contacts[1] = "Jacob - 092";

    for (String c : contacts) {
        System.out.println(c);
    }
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Problem in array: " + e.getMessage());
}
```

2. **Bug Hunter: predict the output before running.** Then run it.

PREDICTION

Your prediction: _____ Actual output: _____

3 A third contact arrives. Do the *exact same four steps* again, starting from where `contacts` left off — new length, new bigger array, copy, place.

MAIN.JAVA

```
int currentLength = contacts.length;
String[] contactsV2 = new String[currentLength + 1];
System.arraycopy(contacts, 0, contactsV2, 0, currentLength);
contactsV2[currentLength] = "Ben - 093";

for (String c : contactsV2) {
    System.out.println(c);
}
```

SCRIBE: WRITE IT DOWN

> **IN YOUR OWN WORDS**
>
> What exactly repeated between step 1 and step 3? Name the four steps:
>
> _____

Picture doing this for a contact list with 4,000 people already in it, one person joining at a time. That repetition — not a crash — is the actual problem collections exist to solve.

PART 2 — ARRAYLIST: THE SAME IDEA, ALREADY BUILT

LIST INTERFACE, ARRAYLIST · 6 MIN

1. Replace the whole array experiment with this. `List` is a contract — anything implementing it must have `add`, `remove`, `get`, `size`. `ArrayList` is one class that honours that contract.

MAIN.JAVA

```
import java.util.ArrayList;
import java.util.List;

List<String> contacts = new ArrayList<>();
contacts.add("Mary - 091");
contacts.add("Jacob - 092");
contacts.add("Ben - 093");

System.out.println(contacts);
```

2. Add a fourth contact past where the raw array would have thrown, then remove one by value.

MAIN.JAVA

```
contacts.add("Fred - 094");
System.out.println("Before deleting Fred: " + contacts);

contacts.remove("Fred - 094");
System.out.println("After deleting Fred: " + contacts);
```

BUG HUNTER

No exception, no manual resizing, no `arraycopy` anywhere. What is `ArrayList` doing internally, every time it runs out of room?

Remember: `List<String>` on the left is the contract; `new ArrayList<>()` on the right is one worker who fulfils it. `<String>` is the diamond operator's partner — it locks the list to one type, so a mistaken `contacts.add(42)` fails at compile time, not three weeks later at runtime.

PART 3 — LINKEDLIST: THE TICKET QUEUE

LINKEDLIST, INSERTION COST · 8 MIN

A `LinkedList` implements the exact same `List` contract as `ArrayList` — but underneath, it's nodes chained together instead of one resizable array.

1. Build a ticket queue. Notice: same `List` interface, only the right-hand side changes.

MAIN.JAVA

```
import java.util.LinkedList;

List<String> queue = new LinkedList<>();
queue.add("Ticket 1");
queue.add("Ticket 3");

System.out.println(queue);
System.out.println("Size: " + queue.size());
```

2. **Predict, then run:** insert "Ticket 2" at index 1, right in the middle.

MAIN.JAVA

```
queue.add(1, "Ticket 2");
System.out.println(queue);
```

THE CINEMA-SEAT PICTURE FROM CLASS

Think of an `ArrayList` as a row of numbered cinema seats: inserting someone in the middle means every person after them physically shifts down one seat. A `LinkedList` is a chain of notes, each one only pointing to the next — inserting means changing where two notes point, and nobody else moves.

3 Do the identical middle-insert on the `contacts` `ArrayList` from Part 2, to see the contract behave the same way on the surface.

MAIN.JAVA

```
contacts.add(1, "Alex - 097");
System.out.println(contacts);
```

SCRIBE: FILL IN FROM THE DISCUSSION

OPERATION	ARRAYLIST	LINKEDLIST
Get by index	_____	_____
Insert/remove in the middle	_____	_____
Insert/remove at the very end	_____	_____

Rule of thumb: default to `ArrayList`. Reach for `LinkedList` only when you can name the reason — frequent inserts/removes in the middle or at the front, like a real queue.

PART 4 — HASHSET: NO MORE DUPLICATE NUMBERS

SET INTERFACE, HASHSET · 7 MIN

1. Reproduce the exact bug from class: an `ArrayList` of phone numbers happily stores the same number twice.

MAIN.JAVA

```
List<String> numbers = new ArrayList<>();
numbers.add("097");
numbers.add("096");
numbers.add("097");   // same number again

System.out.println(numbers);
```

BUG HUNTER

If you already have this contact's number, you should not be able to store it twice. What printed instead?

2. Fix it by swapping the contract: `Set` instead of `List`, `HashSet` instead of `ArrayList`.

MAIN.JAVA

```
import java.util.HashSet;
import java.util.Set;

Set<String> numbers2 = new HashSet<>();
numbers2.add("097");
numbers2.add("096");
numbers2.add("097");

System.out.println(numbers2);
```

The second `"097"` is dropped completely — no exception, no error, it just refuses. `Set` is a contract too: anything implementing it promises no duplicates.

3. Now try it with your own class instead of a plain `String`. Build `Contact.java`:

CONTACT.JAVA

```
public class Contact {
    private String name;
    private String phone;

    public Contact(String name, String phone) {
        this.name = name;
        this.phone = phone;
    }

    public String getName() {
        return name;
    }
}
```

4. **Predict, then run:** add the same-looking `Contact` twice into a `HashSet<Contact>`.

MAIN.JAVA

```
Set<Contact> uniqueContacts = new HashSet<>();
uniqueContacts.add(new Contact("Jacob", "091"));
uniqueContacts.add(new Contact("Jacob", "091"));

System.out.println(uniqueContacts.size());
```

OPEN QUESTION — FLAGGED IN CLASS, NOT YET FIXED

You should see 2, not 1. Two separately-created objects live in two different heap locations, even with identical fields — so `HashSet` correctly treats them as different. Fixing this needs overriding `equals()` and `hashCode()`, which is coming in a later session on overriding. For now, just know *why* it happens.

PART 5 — LINKEDHASHSET AND TREESET

ORDER GUARANTEES ON TOP OF SET · 6 MIN

1. `HashSet` refuses duplicates but makes no promise about order. Prove it, then fix it with `LinkedHashSet`.

MAIN.JAVA

```
import java.util.LinkedHashSet;

Set<String> names = new LinkedHashSet<>();
names.add("Shakil");
names.add("Chanda");
names.add("Ben");
names.add("Jacob");

System.out.println(names);
```

PREDICT BEFORE RUNNING

Still no duplicates allowed — but does the printed order match the order you inserted them? Compare against Part 4's plain `HashSet` output.

2. Now `TreeSet` — a third worker for the same `Set` contract, this one sorts everything automatically.

MAIN.JAVA

```
import java.util.TreeSet;

Set<Integer> numbers3 = new TreeSet<>();
numbers3.add(100);
numbers3.add(50);
numbers3.add(1);
numbers3.add(-10);

System.out.println(numbers3);
```

SCRIBE: THREE SET WORKERS, ONE CONTRACT

CLASS	DUPLICATES	ORDER
`HashSet`	Rejected	_____
`LinkedHashSet`	Rejected	_____
`TreeSet`	Rejected	_____

PART 6 — HASHMAP: THE PHONE BOOK

MAP INTERFACE, KEY-VALUE LOOKUP · 8 MIN

Think about your own phone's Contacts app: you type a name and it jumps straight there, no scrolling. A `Map` is Java's version of that jump.

1. Build a phone book: keys are numbers, values are `Contact` objects.

MAIN.JAVA

```
import java.util.HashMap;
import java.util.Map;

Map<String, Contact> phoneBook = new HashMap<>();
phoneBook.put("097", new Contact("Jacob", "097"));
phoneBook.put("096", new Contact("Mary", "096"));

System.out.println(phoneBook);
```

2. **Predict, then run** — look up a number that was never `put()`:

MAIN.JAVA

```
Contact found = phoneBook.get("095");
System.out.println(found.getName());
```

BUG HUNTER — THIS IS WEEK 3 AGAIN

`get()` on a missing key doesn't throw — it returns `null`. Calling `.getName()` on that `null` gives you a `NullPointerException` right back. Fix the lookup to use a real key ("097") and confirm it now prints `Jacob`.

3. A `Map` isn't one sequence, it's pairs — iterate over its entry set:

MAIN.JAVA

```
for (Map.Entry<String, Contact> entry : phoneBook.entrySet()) {
    System.out.println(entry.getKey() + " -> " +
entry.getValue().getName());
}
```

Same pattern as everywhere else this week: `Map` is the contract, `HashMap` is the worker that fulfils it — `put`, `get`, and `entrySet` are the operations the contract guarantees.

PART 7 — TEST YOUR COLLECTIONS WITH JUNIT

NOT COVERED LIVE — THIS IS YOUR ASSIGNMENT · 5 MIN

The recording didn't get to this part — same Arrange-Act-Assert pattern from Week 2, applied to what you just built.

1. Write these three tests in `ContactManagerTest.java`. Watch each one fail first if you haven't built the matching feature yet, then make it pass.

CONTACTMANAGERTEST.JAVA

```
@Test
void addingContactIncreasesSize() {
    List<Contact> contacts = new ArrayList<>();
    contacts.add(new Contact("Ben", "093"));

    assertEquals(1, contacts.size());
}

@Test
void hashSetRejectsDuplicateNumber() {
    Set<String> numbers = new HashSet<>();
    numbers.add("097");
    numbers.add("097");

    assertEquals(1, numbers.size());
}

@Test
void mapReturnsNullForMissingKey() {
    Map<String, Contact> phoneBook = new HashMap<>();
    phoneBook.put("097", new Contact("Jacob", "097"));

    assertNull(phoneBook.get("000"));
}
```

2. Write a fourth test yourself, with no example to copy: prove that `uniqueContacts.size()` from Part 4 is 2, not 1 — the open question from Part 4, captured as a failing-by-design test.

SCRIBE: WHAT DID TEST 4 PROVE?

> _____

PART 8 — BONUS: A REAL CONTACT FROM THE INTERNET

OPTIONAL — SHOWN LIVE, NOT EXAMINED · 5 MIN

The exact demo from the end of class: a `ContactAPI` class that fetches a real user from a public API and prints the response body.

1. Build `ContactAPI.java`:

CONTACTAPI.JAVA

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class ContactAPI {
    public void fetchContact(int id) {
        try {
            HttpClient client = HttpClient.newHttpClient();
            HttpRequest request = HttpRequest.newBuilder()
                    .uri(URI.create("https://
jsonplaceholder.typicode.com/users/" + id))
                    .GET()
                    .build();

            HttpResponse<String> response =
                    client.send(request,
HttpResponse.BodyHandlers.ofString());

            System.out.println(response.body());
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }
}
```

2 Call it from `Main` and change the `id` to see a different user come back:

MAIN.JAVA

```
ContactAPI api = new ContactAPI();
api.fetchContact(1);
```

Notice the whole thing is wrapped in `try/catch` — a network call is exactly the kind of thing that genuinely fails (no internet, server down), so this is Week 4's exception handling doing real work. For those curious about turning the raw JSON text into a real `Contact` object, look up the `org.json` library mentioned in class.

This is Part Five of the OOP Lab series. Every collection you used today — `List`, `Set`, `Map` — is a contract with more than one worker behind it, and picking the right worker is a design decision, not a syntax choice. Next: what happens when several classes are clearly variations of one idea — inheritance, and the job `protected` was actually built for.